

REPUBLIQUE DU CAMEROUN
Paix-Travail-Patrie

MINISTERE DE L'ENSEIGNEMENT SUPERIEUR

UNIVERSITE D'EBOLWA

FACULTE DES SCIENCES



REPUBLIC OF CAMEROON
Peace-Work-Fatherland

MINISTRY OF HIGHER EDUCATION

THE UNIVERSITY OF EBOLWA

FACULTY OF SCIENCES

EXPOSE SUR LES PATRONS DE CONCEPTION

GROUPE N°1

MEMBRES DU GROUPE

N°	NOMS ET PRENOMS	MATRICULES	FILIERE
1	ABOMO MENGUEME Steve Herman	23I0005FS	TICL2
2	ETO'O ETEME Boris Junior	23I0030FS	TICL2
3	ESSO MINTOUTOU Daniel O'brien	23I0029FS	TICL2

Encadreur : **Dr ADAMOU**

ANNEE ACADEMIQUE

2024/2025

SOMMAIRE

INTRODUCTION.....	1
I-DEFINITION.....	2
II-LES PATRONS DE CREATION.....	2
III- LES PATRONS STRUCTURAUX.....	3
IV- LES PATRONS COMPORTEMENTAUX	4
V- MODELES ASSOCIES AUX PATRONS DE CONCEPTION.....	5
CONCLUSION.....	7

INTRODUCTION

Les patrons de conception (ou **design patterns** en anglais) sont des solutions réutilisables à des problèmes courants qui se posent lors de la conception de logiciels. Ces solutions sont fondées sur des expériences de conception précédentes et permettent de résoudre des problèmes récurrents dans un cadre de développement logiciel, tout en optimisant la qualité du code, sa réutilisabilité, sa flexibilité et sa maintenabilité.

Dans cet exposé, nous explorerons les **patrons de conception** et les modèles associés, en mettant un accent particulier sur le modèle **MVC (Modèle-Vue-Contrôleur)**, qui joue un rôle crucial dans l'architecture des applications modernes.

I- DEFINITION

Un **patron de conception** est une solution réutilisable à un problème récurrent qui survient fréquemment dans le développement logiciel. Plutôt que de réinventer la roue à chaque fois qu'un problème similaire se pose, les développeurs peuvent se référer à ces modèles éprouvés qui proposent des approches standardisées et optimisées pour résoudre ces problèmes.

Les patrons de conception permettent de :

- **Améliorer la réutilisabilité** du code en offrant des solutions génériques qui peuvent être adaptées à différentes situations.
- **Simplifier la maintenance** du logiciel en offrant des structures modulaires, bien définies et faciles à comprendre.
- **Rendre le code plus flexible**, de sorte que les modifications futures soient plus faciles à intégrer.

Les patrons de conception sont classés dans trois grandes catégories :

- **Les patrons de création**
- **Les patrons structuraux**
- **Les patrons comportementaux**

Ces catégories regroupent les patrons selon leur rôle dans l'architecture du logiciel.

II- LES PATRONS DE CREATION

Les **patrons de création** concernent la manière dont les objets sont instanciés et organisés. Ils permettent de créer des objets de manière flexible et de séparer la logique de création d'objets de leur utilisation.

1. Le Patron Singleton

Le patron Singleton garantit qu'une classe n'a qu'une seule instance et offre un point d'accès global à cette instance. Cela est particulièrement utile lorsqu'il faut limiter le nombre d'instances d'une classe.

Exemple : Un gestionnaire de configuration dans une application où une seule instance est nécessaire pour toute l'application.

2. Le Patron Factory Method

Ce patron permet de créer des objets sans spécifier la classe exacte de l'objet qui sera créé. La classe principale délègue la création d'objets à des sous-classes, ce qui permet une grande flexibilité.

Exemple : Une application qui doit instancier différents types de documents (PDF, Word, Excel) en fonction des besoins.

3. Le Patron Abstract Factory

Le patron Abstract Factory permet de créer des familles d'objets associés sans spécifier leurs classes concrètes. Il est utile lorsque plusieurs objets interagissent et doivent être créés dans un contexte cohérent.

Exemple : Un système de gestion d'interface graphique où des objets comme des boutons, des fenêtres et des menus doivent correspondre à un même thème.

4. Le Patron Builder

Le patron Builder permet de construire des objets complexes en séparant la construction de l'objet de sa représentation. Cela rend possible la création d'objets étape par étape.

Exemple : Un système de génération de rapports où différentes parties du rapport (tableau, texte, graphique) sont assemblées de manière ordonnée.

1.5. Le Patron Prototype

Le patron Prototype permet de créer de nouveaux objets en copiant un objet existant (prototypes), au lieu de créer un objet à partir de zéro.

Exemple : Un système où des objets complexes sont clonés pour économiser du temps et des ressources de création.

III- LES PATRONS STRUCTURAUX

Les **patrons structuraux** concernent la manière dont les objets et les classes sont organisés pour créer des structures plus complexes tout en maintenant la flexibilité et la réutilisabilité du code.

1. Le Patron Adapter

Le patron Adapter permet d'adapter l'interface d'une classe existante pour qu'elle soit compatible avec une autre interface, tout en masquant les détails de l'adaptation.

Exemple : Un système qui intègre un module de paiement avec une API différente de celle attendue par le reste de l'application.

2. Le Patron Composite

Le patron Composite permet de composer des objets en structures arborescentes. Il permet de traiter de manière uniforme les objets individuels et les compositions d'objets.

Exemple : La gestion d'un système de fichiers où à la fois les fichiers et les répertoires sont représentés par des objets, et un répertoire peut contenir d'autres répertoires ou des fichiers.

3. Le Patron Proxy

Le patron Proxy permet de représenter un objet par un substitut afin de contrôler l'accès à l'objet réel, souvent pour des raisons de performance ou de sécurité.

Exemple : Utilisation d'un proxy pour accéder à une base de données distante, où le proxy gère la mise en cache.

4. Le Patron Facade

Le patron Facade offre une interface simplifiée à un sous-système complexe, en masquant les détails d'implémentation et en réduisant les dépendances externes.

Exemple : Un système bancaire qui fournit une interface unique pour gérer les comptes, les prêts et les transactions.

5. Le Patron Flyweight

Le patron Flyweight permet de réduire la consommation de mémoire en partageant des objets qui possèdent un état identique au lieu de créer de nouvelles instances.

Exemple : Un éditeur de texte où les caractères sont partagés entre les différents documents pour économiser de la mémoire.

IV- LES PATRONS COMPORTEMENTAUX

Les **patrons comportementaux** se concentrent sur la manière dont les objets interagissent entre eux et sur la gestion des comportements des objets dans un système.

1. Le Patron Strategy

Le patron Strategy permet de définir une famille d'algorithmes, de les encapsuler et de les rendre interchangeables, permettant ainsi de modifier le comportement d'un objet à l'exécution.

Exemple : Un système de paiement où le mode de paiement (carte de crédit, PayPal, virement bancaire) peut être modifié dynamiquement.

2. Le Patron Observer

Le patron Observer définit une relation de dépendance entre objets, où un changement d'état dans un objet entraîne une notification des objets dépendants (les observateurs).

Exemple : Un système de notifications où plusieurs utilisateurs doivent être avertis lorsqu'un événement se produit (par exemple, mise à jour de contenu).

3. Le Patron Command

Le patron Command permet d'encapsuler une requête sous forme d'objet. Cela permet de paramétrer des objets avec des demandes, de les mettre en file d'attente ou de les annuler.

Exemple : Dans une interface graphique, chaque action (cliquer sur un bouton, sélectionner un élément) est représentée par un objet commande.

4. Le Patron Iterator

Le patron Iterator permet de traverser une collection d'objets sans exposer sa structure interne, simplifiant ainsi les opérations de parcours.

Exemple : Un système où l'on souhaite itérer sur une collection de fichiers sans se soucier du type de collection.

5. Le Patron State

Le patron State permet à un objet de changer son comportement lorsqu'il change d'état. Cela permet de gérer les transitions d'états de manière plus claire et plus flexible.

Exemple : Modélisation des différents états d'une commande dans un système de gestion d'e-commerce (en attente, en traitement, expédiée, livrée).

V- MODELES ASSOCIES AUX PATRONS DE CONCEPTION

Outre les patrons de conception traditionnels, il existe plusieurs modèles et architectures qui peuvent être associés à ces patrons pour améliorer la conception des systèmes logiciels. L'un des modèles les plus populaires et utilisés dans le développement d'applications modernes est le modèle **MVC** (Modèle-Vue-Contrôleur).

❖ Le Modèle MVC (Modèle-Vue-Contrôleur)

Le **modèle MVC** est un patron architectural qui sépare une application en trois composants principaux :

- **Modèle (Model)** : Représente les données et la logique métier de l'application. C'est la partie qui manipule les informations et effectue des calculs.
- **Vue (View)** : Responsable de l'affichage des données et de l'interface utilisateur. Elle reflète l'état du modèle, mais elle ne contient pas de logique métier.
- **Contrôleur (Controller)** : Gère les interactions utilisateur, reçoit les entrées et effectue les actions appropriées sur le modèle et la vue. Le contrôleur fait le lien entre le modèle et la vue, traitant la logique d'application.

Avantages du MVC :

- Séparation claire des préoccupations, ce qui rend l'application plus facile à maintenir et à étendre.
- Possibilité de modifier indépendamment l'interface utilisateur (vue) sans toucher à la logique métier (modèle), et vice versa.
- Supporte le développement collaboratif, car les développeurs peuvent travailler sur différentes parties (modèle, vue, contrôleur) de manière indépendante.

Exemple : Les frameworks web comme **Ruby on Rails**, **Angular**, **Spring MVC** ou **Django** utilisent l'architecture MVC pour structurer les applications web de manière modulaire et efficace.

❖ Autres modèles

- **Refactoring** : Processus de restructuration du code pour améliorer sa lisibilité et sa maintenabilité sans en changer le comportement. Le refactoring peut introduire des patrons de conception pour améliorer la structure du code.
- **Conception par Contrat (Design by Contract)** : Ce modèle met l'accent sur les préconditions, postconditions et invariants lors de la conception des interfaces entre les classes et les objets.
- **Test-Driven Development (TDD)** : La pratique de l'écriture de tests avant de coder peut inciter à l'application de certains patrons, en particulier ceux liés à la séparation des préoccupations et à la flexibilité du code.

CONCLUSION

Les patrons de conception sont des outils puissants qui permettent de résoudre des problèmes récurrents de manière structurée et réutilisable. L'utilisation de ces patrons favorise la création de logiciels robustes, flexibles et faciles à maintenir. En complément des patrons traditionnels, des modèles architecturaux comme **MVC** jouent un rôle essentiel dans la gestion de l'interaction entre les différentes parties d'une application. Adopter ces concepts dans le développement logiciel permet de créer des systèmes plus performants, modulaires et adaptés aux besoins changeants des utilisateurs et des entreprises.